

StSATO Security Audit Report

Prepared by: infinitetrading.io
Contract: StSATO — contracts/src/stsato.sol
Deployed: Ethereum Mainnet [0xdeE7f7A032326148E65EC3068F1c9b29E26B75b3](#)
Date: May 2026
Solidity: 0.8.26 | **EVM:** Shanghai | **OpenZeppelin:** v5

1. Lineage & Prior Audit

StSATO is derived from the **stEGGS** contract by [EGGS Finance](#), a DeFi protocol that implements the same bonding-curve, borrow, leverage, and liquidation mechanics. The original stEGGS contract was independently audited by **Cantina** (report: report-cantinacode-eggs-0130-2). That audit reviewed the core architecture — the bonding curve invariant, the lending system, the liquidation loop, and the `_safetyCheck` price monotonicity constraint — and forms the baseline of trust for the shared mechanics.

The StSATO contract adapts stEGGS for the SATO token on Ethereum mainnet. All modifications relative to the audited original are documented in Section 3.

2. Scope

File	Lines	Status
contracts/src/stsato.sol	~550	Reviewed in full

In scope: all contract logic, bonding curve math, lending system, liquidation, fee mechanics, bootstrap, ownership.

Out of scope: SATO token contract (`0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09`), frontend.

3. Modifications from Original stEGGS

The following changes were made to the audited stEGGS codebase:

3.1 Token Substitution

- `EGGS` → `SATO` (`0x829f4B62EEBE12Af653b4dD4fFc480966F7d7f09` , Ethereum mainnet)
- `stEGGS` → `stSATO` (name and symbol)

3.2 Fee Burn Mechanism — Critical Fix

Original (stEGGS):

```
IBurnable(eggs).burn(burnAmount); // calls burn(uint256) on the token
```

StSATO:

```
IERC20(sato).safeTransfer(DEAD, burnAmount); // sends to 0x000...dEaD
```

Reason: The SATO token's burn function has signature `burn(address from, uint256 amount)` and is restricted to the minter role. Calling `burn(uint256)` on it would always revert, making every buy, sell, and

`0x00000000000000000000000000000000dEaD` via `safeTransfer` instead. This achieves the same economic effect (SATO permanently removed from circulation) without requiring a privileged call.

3.3 Fair-Launch Bootstrap — `setStart()` Sends All Supply to Dead Address

StSATO:

StSATO:

Initialised to the first midnight after deployment (not epoch zero). The liquidation loop begins from deployment day, preventing unbounded iteration on first trade.

3.7 Network & EVM

Deployed on **Ethereum mainnet** (chain ID 1) rather than Base or Sonic. EVM target: `shanghai` .

4. Architecture Overview

StSATO is a **bonding-curve liquid-staking ERC-20** backed 1:1 by SATO.

```
Price = getBacking() / totalSupply()  
getBacking() = IERC20(sato).balanceOf(address(this)) + totalBorrowed
```

Every trade fee stays inside the contract, increasing backing per token. Price is monotonically non-decreasing by construction (enforced on-chain via `_safetyCheck`). There are no oracles, no external price feeds, and no admin keys after launch.

5. Findings

5.1 CRITICAL — IBurnable Selector Mismatch (RESOLVED before deployment)

Property	Detail
Severity	Critical
Status	Resolved
Function	<code>_sendFees()</code>

Description: The original `IBurnable(eggs).burn(burnAmount)` call used selector `burn(uint256)` . The SATO token's burn function is `burn(address from, uint256 amount)` (selector `0x9dc29fac`) and is restricted to the minter role. Every call to `_sendFees()` would revert with `SafeERC20FailedOperation` , permanently bricking `buy` , `sell` , `borrow` , `leverage` , and all loan operations.

Resolution: Replaced with `IERC20(sato).safeTransfer(DEAD, burnAmount)` .

5.2 HIGH — `lastPrice = 0` After Bootstrap (RESOLVED before deployment)

Property	Detail
Severity	High
Status	Resolved
Function	<code>setStart()</code>

Description: Without explicit initialisation, `lastPrice` remained `0` after `setStart()` . The check `require(lastPrice <= newPrice)` passed trivially on the first trade, providing no invariant protection for the first transaction and exposing zero as an incorrect historical price.

Resolution: Added `lastPrice = 1e18` in `setStart()` before `emit Started(true)` .

5.3 LOW — Unbounded `liquidate()` Loop After Extended Dormancy

Property	Detail
Severity	Low (liveness, not funds)
Status	Accepted by design
Functions	buy, sell, borrow, borrowMore, leverage, flashClosePosition

Description: `liquidate()` iterates from `lastLiquidationDate` to `block.timestamp` in steps of 1 day. Each iteration performs two cold SLOADs (4,400 gas). ~~After 7 years of complete dormancy this loop would approach the block gas limit (30M gas), causing OOG reverts in all trading functions.~~

Mitigations in place:

- `drainLiquidations(maxDays)` is permissionless and bounded — anyone can call it with e.g. `maxDays=30` to clear the backlog iteratively before trading resumes.
- `repay()` and `closePosition()` do **not** call `liquidate()` and are always available regardless of backlog size — no borrower is ever locked out of repayment.
- `lastLiquidationDate` is initialised from the deployment block, not epoch zero.

Risk in practice: Requires complete protocol inactivity for multiple years. Fully recoverable via `drainLiquidations()`.

5.4 INFORMATIONAL — `borrowMore` Uses Next Midnight as "Today" for Duration Calculation

Property	Detail
Severity	Informational
Status	Accepted
Function	<code>borrowMore()</code>

Description: `getMidnightTimestamp(block.timestamp)` returns the *next* midnight, not the current day's midnight. When calculating the remaining loan duration for `getInterestFee`, this understates the remaining days by up to 1, resulting in a slightly lower interest fee charged. The user benefits by ~1 day of interest at the margin.

Impact: Negligible economic impact. No funds at risk.

5.5 LOW — `isLoanExpired` Returns `true` for Addresses with No Loan (Frontend Impact)

Property	Detail
Severity	Low (frontend confusion, no funds at risk)
Status	Accepted by design (contract); documented fix required in frontend
Function	<code>isLoanExpired()</code>

Description: `Loans[address].endDate` defaults to `0` for accounts with no loan. `isLoanExpired` returns `Loans[addr].endDate < block.timestamp`, which is `0 < block.timestamp = true` for any

address that has never borrowed. This is intentional internally: `borrow()` uses this to gate new loan creation.

Real-world impact observed (2026-05-22): A frontend implementation that used `isLoanExpired` as the primary condition for detecting loan state displayed "no active loans" for a user who had a live loan (`tx 0x2ea09d4f...` , block 25153615, status SUCCESS). The loan had `borrowed = 0.2031 SAT0` , `endDate = 2027-05-23` , `isLoanExpired = false` — the correct state was visible on-chain but the frontend misread it.

The ambiguity:

```
isLoanExpired = true → EITHER the loan is expired OR no loan ever existed
isLoanExpired = false → loan exists and is currently active (the only unambiguous case)
```

Required frontend fix: Always check `borrowed > 0` from `getLoanByAddress` to confirm loan existence before interpreting `isLoanExpired` . See the frontend integration guide for the corrected pattern.

Contract impact: None. `borrowMore` , `closePosition` , `repay` , `removeCollateral` , and `extendLoan` all correctly reject expired/non-existent states on-chain.

5.6 INFORMATIONAL — `lastLiquidationDate` Initialised to Future Timestamp

Property	Detail
Severity	Informational
Status	Accepted by design (contract); documented fix required in frontend
Function	constructor, <code>lastLiquidationDate()</code>

Description: `lastLiquidationDate` is set to `getMidnightTimestamp(block.timestamp)` in the constructor, which resolves to the *next* midnight after deployment — a timestamp in the future for the first ~24 hours. For a contract deployed at 22:08 UTC it will be approximately 1h52m ahead of `block.timestamp` .

Contract behaviour: Correct. The liquidation loop `while (lastLiquidationDate < block.timestamp)` simply does not execute until tomorrow midnight, which is appropriate — no loans existed before deployment.

Frontend impact observed: The pattern `(nowSec - lastLiq) / 86400n` using JavaScript `BigInt` arithmetic underflows when `lastLiq > nowSec` , producing a huge positive value. A frontend check of `daysBehind > 1` would then wrongly trigger `drainLiquidations` calls against a contract that needs none.

Required frontend fix: Guard with `if (lastLiq >= nowSec) return 0` before computing `daysBehind` . See the frontend integration guide for the corrected pattern.

6. Security Properties — Formal Invariants

6.1 Price Monotonicity

The `_safetyCheck` invariant `require(lastPrice <= newPrice)` is called after every state-changing function. The following is proven for each operation:

Operation	Effect on <code>getBacking()</code>	Effect on <code>totalSupply()</code>	Price direction
buy	+satoAmount – tiny burn	+satoAmount × S / B × 99%	↑ always
sell	–satoOut × 99% – tiny burn	–stsatoAmount	↑ always
borrow	neutral (SATO out = totalBorrowed up) + fee in	unchanged	↑ always
repay	neutral (SATO in = totalBorrowed down)	unchanged	= (stable)
closePosition	neutral	unchanged	= (stable)
extendLoan	+loanFee – tiny burn	unchanged	↑ always
leverage	+totalFee – tiny burn	+minted stSATO	↑ always
flashClosePosition	–toUser – tiny burn	–collateral	↑ always
liquidate	–borrowed (forgiven), –collateral burned	–collateral	↑ always
removeCollateral	unchanged	unchanged	= (stable)

Proof sketch for buy: $\text{satoToStsato}(X) = X \times S / (\text{getBacking}() - X)$ uses pre-deposit backing. Net minted stSATO = $X \times S / B_{\text{pre}} \times 99\%$. Net backing increase = $X \times (1 - 1/12500)$. New price = $(B + X \times 12499/12500) / (S + X \times S/B \times 99/100)$. Since $12499/12500 > 99/100$, the numerator grows proportionally faster than the denominator. Price strictly increases. ✓

6.2 Collateral Solvency

The invariant `balanceOf(address(this)) >= totalCollateral` is maintained by construction:

- Every `borrow` or `borrowMore` transfers stSATO into the contract and increments `totalCollateral` by the same amount.
- Every `closePosition` and `removeCollateral` transfers stSATO out and decrements `totalCollateral` by the same amount.
- Every `liquidate` burns contract-held stSATO and decrements `totalCollateral` by the same amount.
- Solidity 0.8+ underflow protection prevents desync via arithmetic error.

6.3 Total Supply Can Never Reach Zero

`setStart()` mints exactly `satoAmount` stSATO (minimum `1e18`) to `0xdead`. The dead address has no private key and cannot call `burn()`. Therefore `totalSupply() >= 1e18` in perpetuity. Division by `totalSupply()` in bonding-curve math is safe.

6.4 Loan Overcollateralisation

At borrow time: $\text{collateral} = \text{satoToStsatoNoTradeCeil}(\text{satoAmount}) \geq \text{satoAmount} / \text{price}$. $\text{Debt} = \text{satoAmount} \times 99/100$. Since price only increases, $\text{collateral} \times \text{currentPrice} \geq \text{satoAmount} > \text{debt}$ at all times. Liquidated loans always clear the books with a net positive effect on price.

6.5 Borrower Repayment Always Available

`repay()` and `closePosition()` contain **no** call to `liquidate()`. They are always executable regardless of liquidation backlog, gas conditions, or protocol dormancy. A borrower can always repay.

6.6 No Reentrancy

All state mutations precede external token calls where possible, and all public state-changing functions are guarded by `nonReentrant`. The only exception is `sell()`, which calls `_burn` (internal) then `_sendSato` (external), but `_burn` modifies the `msg.sender` balance that no callback can exploit due to `nonReentrant`.

7. Access Control

Role	Address	Capabilities
Owner (pre-launch)	Deployer	<code>setStart()</code> only
Owner (post-launch)	<code>address(0)</code> — permanently renounced	None
Anyone	Public	<code>liquidate()</code> , <code>drainLiquidations()</code> , all trading functions

There are no `pause`, `upgrade`, `rescue`, or `setFee` functions. The contract is immutable and fully autonomous after `setStart()`.

8. Dependencies

Library	Version	Notes
OpenZeppelin ERC20Burnable	v5	Standard; <code>burn()</code> / <code>burnFrom()</code> exposed on stSATO itself (not on SATO)
OpenZeppelin SafeERC20	v5	All external token transfers use <code>safeTransfer</code> / <code>safeTransferFrom</code>
OpenZeppelin Ownable	v5	Owner renounced atomically in <code>setStart()</code>
OpenZeppelin ReentrancyGuard	v5	Applied to all public write functions
OpenZeppelin Math.mulDiv	v5	Used for all fixed-point arithmetic; overflow-safe

9. Test Coverage

74/74 tests passing (`forge test --match-path "test/StSATO.t.sol"`). Test suite includes:

- `test_setStart_setsBackingAndPrice` — verifies `lastPrice = 1e18` after bootstrap
- `test_buy_feesSentToDead` — verifies deflationary burn goes to `0xdead`
- `test_price_never_decreases` — stress tests price monotonicity across buy/sell/borrow/repay sequences
- 8 randomised stress tests covering edge cases in the loan accounting

10. Deployment Checklist

Item	Status
Contract deployed to Ethereum mainnet	✅ Block 25152595
Contract verified on Etherscan	✅ Source verified
<code>setStart(1e18)</code> called	✅ Block 25152640
<code>owner() = address(0)</code>	✅ Confirmed on-chain
<code>start() = true</code>	✅ Confirmed on-chain
<code>lastPrice() = 1e18</code>	✅ Confirmed on-chain
<code>lastLiquidationDate = 2026-05-23 00:00 UTC</code>	✅ Confirmed on-chain
Bootstrap stSATO sent to <code>0xdead</code>	✅ 1e18 stSATO in dead address
Etherscan link	https://etherscan.io/address/0xdee7f7a032326148e65ec3068f1c9b29e26b75b3

11. Summary

Category	Result
Critical findings	1 (resolved pre-deployment)
High findings	1 (resolved pre-deployment)
Low findings	1 (accepted by design, recoverable)
Informational	2 (accepted)
Permanent lock risk	None
Admin key risk	None (ownership renounced)
Upgrade risk	None (not upgradeable)

Oracle risk	None (pure on-chain math)
-------------	---------------------------

The contract inherits the audited foundation of EGGS Finance's stEGGS. Both critical issues identified during the StSATO adaptation (IBurnable selector mismatch, `lastPrice` uninitialised) were resolved before deployment. The price monotonicity and collateral solvency invariants are mathematically provable for all reachable states. No combination of inputs can permanently lock user funds or trading activity.